



Graph Neural Networks Theory to Practice

Emanuel Sallinger



Popular platforms for graph computing.

Platform	Link	Reference
PyTorch Geometric	https://github.com/rusty1s/pytorch_geometric	Fey and Lenssen (2019)
Deep Graph Library	https://github.com/dmlc/dgl	Wang et al. (2019b)
AliGraph	https://github.com/alibaba/aligraph	Zhu et al. (2019a)
GraphVite	https://github.com/DeepGraphLearning/graphvite	Zhu et al. (2019b)
Paddle Graph Learning	https://github.com/PaddlePaddle/PGL	
Euler	https://github.com/alibaba/euler	
Plato	https://github.com/tencent/plato	
CogDL	https://github.com/THUDM/cogdl/	
OpenNE	https://github.com/thunlp/OpenNE/tree/pytorch	

Model	Link
GGNN (2015)	https://github.com/yujiali/ggnn
Neurals FPs (2015)	https://github.com/HIPS/neural-fingerprint
ChebNet (2016)	https://github.com/mdeff/cnn_graph
DNGR (2016)	https://github.com/ShelsonCao/DNGR
SDNE (2016)	https://github.com/suanrong/SDNE
GAE (2016)	https://github.com/limaosen0/Variational-Graph-Auto-Encoders
DRNE (2016)	https://github.com/tadpole/DRNE
Structural RNN (2016)	https://github.com/asheshjain399/RNNexp
DCNN (2016)	https://github.com/jcatw/dcn
GCN (2017)	https://github.com/tkipf/gcn
CayleyNet (2017)	https://github.com/amoliu/CayleyNet
GraphSage (2017)	https://github.com/williamleif/GraphSAGE
GAT (2017)	https://github.com/PetarV-/GAT
CLN (2017)	https://github.com/trangptm/Column_networks
ECC (2017)	https://github.com/mys007/ecc
MPNNs (2017)	https://github.com/brain-research/mpnn
MoNet (2017)	https://github.com/pierrebaque/GeometricConvolutionsBench

Source: <https://arxiv.org/ftp/arxiv/papers/1812/1812.08434.pdf>



**PyTorch**
geometric

latest

Search docs

NOTES

- Installation
- Introduction by Example
- Creating Message Passing Networks
- Creating Your Own Datasets
- Advanced Mini-Batching
 - Memory-Efficient Aggregations
- TorchScript Support
 - Colab Notebooks
 - External Resources

PACKAGE REFERENCE

- torch_geometric
- torch_geometric.nn
- torch_geometric.data
- torch_geometric.datasets
- torch_geometric.transforms
- torch_geometric.utils
- torch_geometric.io

» PyTorch Geometric Documentation

Edit on GitHub

PYTORCH GEOMETRIC DOCUMENTATION

PyTorch Geometric is a geometric deep learning extension library for [PyTorch](#).

It consists of various methods for deep learning on graphs and other irregular structures, also known as [geometric deep learning](#), from a variety of published papers. In addition, it consists of an easy-to-use mini-batch loader for many small and single giant graphs, a large number of common benchmark datasets (based on simple interfaces to create your own), and helpful transforms, both for learning on arbitrary graphs as well as on 3D meshes or point clouds.

Notes

- [Installation](#)
- [Introduction by Example](#)
- [Creating Message Passing Networks](#)
- [Creating Your Own Datasets](#)
- [Advanced Mini-Batching](#)
- [Memory-Efficient Aggregations](#)
- [TorchScript Support](#)
- [Colab Notebooks](#)
- [External Resources](#)

Package Reference

- [torch_geometric](#)
- [torch_geometric.nn](#)
- [torch_geometric.data](#)
- [torch_geometric.datasets](#)

PyTorch
geometric

latest

Search docs

NOTES

⊞ Installation

⊞ Introduction by Example

⊞ Creating Message Passing Networks

The "MessagePassing" Base Class

Implementing the GCN Layer

Implementing the Edge Convolution

⊞ Creating Your Own Datasets

⊞ Advanced Mini-Batching

Memory-Efficient Aggregations

[🏠](#) » Creating Message Passing Networks[🔗 Edit on GitHub](#)

CREATING MESSAGE PASSING NETWORKS

Generalizing the convolution operator to irregular domains is typically expressed as a *neighborhood aggregation* or *message passing* scheme. With $\mathbf{x}_i^{(k-1)} \in \mathbb{R}^F$ denoting node features of node i in layer $(k-1)$ and $\mathbf{e}_{j,i} \in \mathbb{R}^D$ denoting (optional) edge features from node j to node i , message passing graph neural networks can be described as

$$\mathbf{x}_i^{(k)} = \gamma^{(k)} \left(\mathbf{x}_i^{(k-1)}, \square_{j \in \mathcal{N}(i)} \phi^{(k)} \left(\mathbf{x}_i^{(k-1)}, \mathbf{x}_j^{(k-1)}, \mathbf{e}_{j,i} \right) \right),$$

where $\square$ denotes a differentiable, permutation invariant function, e.g., sum, mean or max, and γ and ϕ denote differentiable functions such as MLPs (Multi Layer Perceptrons).

- [The "MessagePassing" Base Class](#)
- [Implementing the GCN Layer](#)
- [Implementing the Edge Convolution](#)



update the state **aggregate** the messages **messages** from neighbours



$$H_v^{t+1} = u(H_v^t, a_{u \in N(v)}(m(H_v^t, H_u^t, X_{u,v})))$$

⊕ Introduction by Example

⊖ Creating Message Passing Networks

The "MessagePassing" Base Class

Implementing the GCN Layer

Implementing the Edge Convolution

⊕ Creating Your Own Datasets

⊕ Advanced Mini-Batching

Memory-Efficient Aggregations

$$\mathbf{x}_i^{(k)} = \gamma^{(k)} \left(\mathbf{x}_i^{(k-1)}, \square_{j \in \mathcal{N}(i)} \phi^{(k)} \left(\mathbf{x}_i^{(k-1)}, \mathbf{x}_j^{(k-1)}, \mathbf{e}_{j,i} \right) \right),$$

where $\square$ denotes a differentiable, permutation invariant function, e.g., sum, mean or max, and γ and ϕ denote differentiable functions such as MLPs (Multi Layer Perceptrons).

- The "MessagePassing" Base Class
- Implementing the GCN Layer
- Implementing the Edge Convolution



```
1. Introduction.ipynb
File Edit View Insert Runtime Tools Help Changes will not be saved

+ Code + Text Copy to Drive

# Install required packages.
!pip install -q torch-scatter -f https://pytorch-geometric.com/whl/torch-1.8.0+cu101.html
!pip install -q torch-sparse -f https://pytorch-geometric.com/whl/torch-1.8.0+cu101.html
!pip install -q torch-geometric

# Helper function for visualization.
%matplotlib inline
import torch
import networkx as nx
import matplotlib.pyplot as plt

def visualize(h, color, epoch=None, loss=None):
    plt.figure(figsize=(7,7))
    plt.xticks([])
    plt.yticks([])

    if torch.is_tensor(h):
        h = h.detach().cpu().numpy()
        plt.scatter(h[:, 0], h[:, 1], s=140, c=color, cmap="Set2")
        if epoch is not None and loss is not None:
            plt.xlabel(f'Epoch: {epoch}, Loss: {loss.item():.4f}', fontsize=16)
    else:
        nx.draw_networkx(G, pos=nx.spring_layout(G, seed=42), with_labels=False,
                        node_color=color, cmap="Set2")
    plt.show()

51kB 2.5MB/s
Building wheel for torch-geometric (setup.py) ... done
```

Introduction: Hands-on Graph Neural Networks

Recently, deep learning on graphs has emerged to one of the hottest research fields in the deep learning community. Here, **Graph Neural Networks (GNNs)** aim to generalize classical deep learning concepts to irregular structured data (in contrast to images or texts) and to enable neural networks to reason about objects and their relations.

This is done by following a simple **neural message passing scheme**, where node features $\mathbf{x}_v^{(\ell)}$ of all nodes $v \in \mathcal{V}$ in a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ are iteratively updated by aggregating localized information from their neighbors $\mathcal{N}(v)$:

$$\mathbf{x}_v^{(\ell+1)} = f_{\theta}^{(\ell+1)} \left(\mathbf{x}_v^{(\ell)}, \left\{ \mathbf{x}_w^{(\ell)} : w \in \mathcal{N}(v) \right\} \right)$$





PyTorch geometric

latest

NOTES

- Installation
- Introduction by Example
- Creating Message Passing Networks
- Creating Your Own Datasets
- Advanced Mini-Batching
- Memory-Efficient Aggregations
- TorchScript Support
- Colab Notebooks
- External Resources

PACKAGE REFERENCE

torch_geometric

torch_geometric.nn

- Convolutional Layers
- Dense Convolutional Layers
- Normalization Layers
- Global Pooling Layers
- Pooling Layers
- Dense Pooling Layers
- Unpooling Layers
- Models**

Models

ARGA	The Adversarially Regularized Graph Auto-Encoder model from the "Adversarially Regularized Graph Autoencoder for Graph Embedding" paper.
ARGVA	The Adversarially Regularized Variational Graph Auto-Encoder model from the "Adversarially Regularized Graph Autoencoder for Graph Embedding" paper.
AttentiveFP	The Attentive FP model for molecular representation learning from the "Pushing the Boundaries of Molecular Representation for Drug Discovery with the Graph Attention Mechanism" paper, based on graph attention mechanisms.
CorrectAndSmooth	The correct and smooth (C&S) post-processing model from the "Combining Label Propagation And Simple Models Out-performs Graph Neural Networks" paper, where soft predictions $\mathbf{Z}$ (obtained from a simple base predictor) are first corrected based on ground-truth training label information $\mathbf{Y}$ and residual propagation
DeepGCNLayer	The skip connection operations from the "DeepGCNs: Can GCNs Go as Deep as CNNs?" and "All You Need to Train Deeper GCNs" papers.
DeepGraphInfomax	The Deep Graph Infomax model from the "Deep Graph Infomax" paper based on user-defined encoder and summary model $\mathcal{E}$ and $\mathcal{R}$ respectively, and a corruption function $\mathcal{C}$.
DimeNet	The directional message passing neural network (DimeNet) from the "Directional Message Passing for Molecular Graphs" paper.
GAE	The Graph Auto-Encoder model from the "Variational Graph Auto-Encoders" paper based on user-defined encoder and decoder models.
GNNExplainer	The GNN-Explainer model from the "GNNExplainer: Generating Explanations for Graph Neural Networks" paper for identifying compact subgraph structures and small subsets node features that play a crucial role in a GNN's node-predictions.
GraphUNet	The Graph U-Net model from the "Graph U-Nets" paper which implements a U-Net like architecture with graph pooling and unpooling operations.
InnerProductDecoder	The inner product decoder from the "Variational Graph Auto-Encoders" paper



```
CLASS SGConv ( in_channels: int, out_channels: int, K: int = 1, cached: bool = False, add_self_loops: bool = True, bias: bool = True, **kwargs ) [source]
```

The simple graph convolutional operator from the “Simplifying Graph Convolutional Networks” paper

$$\mathbf{X}' = \left(\hat{\mathbf{D}}^{-1/2} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-1/2} \right)^K \mathbf{X} \Theta,$$

where $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ denotes the adjacency matrix with inserted self-loops and $\hat{D}_{ii} = \sum_{j=0} \hat{A}_{ij}$ its diagonal degree matrix. The adjacency matrix can include other values than `1` representing edge weights via the optional `edge_weight` tensor.

PARAMETERS:

- `in_channels` (*int*) – Size of each input sample.
- `out_channels` (*int*) – Size of each output sample.
- `K` (*int, optional*) – Number of hops K . (default: `1`)
- `cached` (*bool, optional*) – If set to `True`, the layer will cache the computation of $\left(\hat{\mathbf{D}}^{-1/2} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-1/2} \right)^K \mathbf{X}$ on first execution, and will use the cached version for further executions. This parameter should only be set to `True` in transductive learning scenarios. (default: `False`)
- `add_self_loops` (*bool, optional*) – If set to `False`, will not add self-loops to the input graph. (default: `True`)
- `bias` (*bool, optional*) – If set to `False`, the layer will not learn an additive bias. (default: `True`)
- `**kwargs` (*optional*) – Additional arguments of `torch_geometric.nn.conv.MessagePassing`.



```
CLASS SAGEConv ( in_channels: Union[int, Tuple[int, int]], out_channels: int, normalize: bool = False,  
root_weight: bool = True, bias: bool = True, **kwargs ) [source]
```

The GraphSAGE operator from the “Inductive Representation Learning on Large Graphs” paper

$$\mathbf{x}'_i = \mathbf{W}_1 \mathbf{x}_i + \mathbf{W}_2 \cdot \text{mean}_{j \in \mathcal{N}(i)} \mathbf{x}_j$$

PARAMETERS:

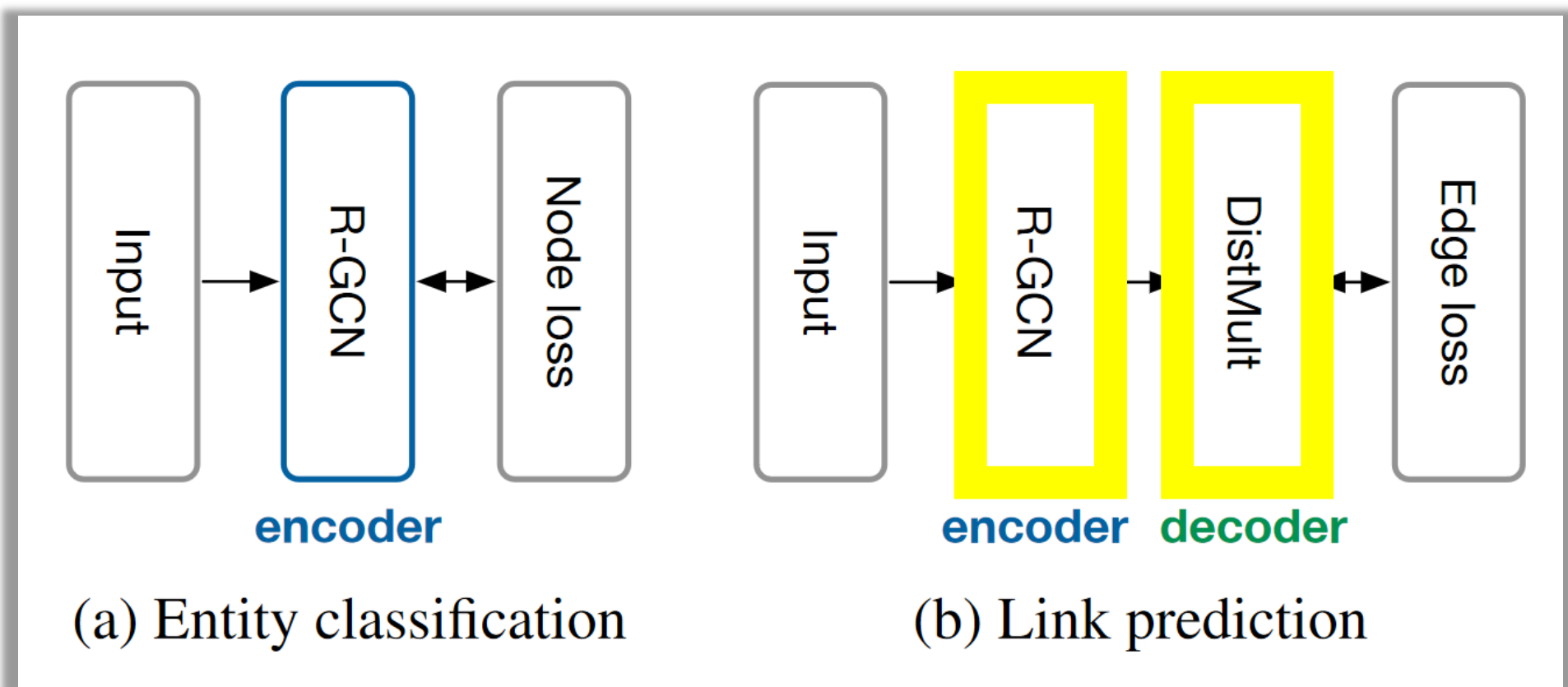
- **in_channels** (*int* or *tuple*) – Size of each input sample. A tuple corresponds to the sizes of source and target dimensionalities.
- **out_channels** (*int*) – Size of each output sample.
- **normalize** (*bool*, *optional*) – If set to `True`, output features will be ℓ_2 -normalized, i.e., $\frac{\mathbf{x}'_i}{\|\mathbf{x}'_i\|_2}$. (default: `False`)
- **root_weight** (*bool*, *optional*) – If set to `False`, the layer will not add transformed root node features to the output. (default: `True`)
- **bias** (*bool*, *optional*) – If set to `False`, the layer will not learn an additive bias. (default: `True`)
- ****kwargs** (*optional*) – Additional arguments of `torch_geometric.nn.conv.MessagePassing`.



Link Prediction



Link Prediction







PyTorch

geometric

latest

NOTES

- Installation
- Introduction by Example
- Creating Message Passing Networks
- Creating Your Own Datasets
- Advanced Mini-Batching
 - Memory-Efficient Aggregations
- TorchScript Support
 - Colab Notebooks
 - External Resources

PACKAGE REFERENCE

- torch_geometric
 - torch_geometric.nn
 - Convolutional Layers
 - Dense Convolutional Layers
 - Normalization Layers

```
class RGCNConv ( in_channels: Union[int, Tuple[int, int]], out_channels: int, num_relations: int, num_bases: Optional[int] = None, num_blocks: Optional[int] = None, aggr: str = 'mean', root_weight: bool = True, bias: bool = True, **kwargs ) [source]
```

The relational graph convolutional operator from the “Modeling Relational Data with Graph Convolutional Networks” paper

$$\mathbf{x}'_i = \Theta_{\text{root}} \cdot \mathbf{x}_i + \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \frac{1}{|\mathcal{N}_r(i)|} \Theta_r \cdot \mathbf{x}_j,$$

where $\mathcal{R}$ denotes the set of relations, i.e. edge types. Edge type needs to be a one-dimensional `torch.long` tensor which stores a relation identifier $\in \{0, \dots, |\mathcal{R}| - 1\}$ for each edge.

Note

This implementation is as memory-efficient as possible by iterating over each individual relation type. Therefore, it may result in low GPU utilization in case the graph has a large number of relations. As an alternative approach, `FastRGCNConv` does not iterate over each individual type, but may consume a large amount of memory to compensate. We advise to check out both implementations to see which one fits your needs.

PARAMETERS:

- `in_channels` (*int or tuple*) – Size of each input sample. A tuple corresponds to the sizes of source and target dimensionalities. In case no input features are given, this argument should correspond to the number of nodes in your graph.
- `out_channels` (*int*) – Size of each output sample.
- `num_relations` (*int*) – Number of relations.
- `num_bases` (*int, optional*) – If set to not `None`, this layer will use the basis-decomposition regularization scheme where `num_bases` denotes the number of bases to use. (default: `None`)



DGL
0.4.x

Search docs

INSTALLATION

Install DGL

Working with different backends

GET STARTED

DGL at a Glance

DGLGraph and Node/edge Features

Message Passing Tutorial

Graph Classification Tutorial

Working with Heterogeneous Graphs

TUTORIALS

Graph neural networks and its variants

Graph Convolutional Network

Relational graph convolutional network

A brief introduction to R-GCN

Key ideas of R-GCN

Implement R-GCN in DGL

R-GCN layers

Full R-GCN model defined

Handle dataset

Create graph and model

Training loop

The second task, link prediction

Docs » Graph neural networks and its variants » Relational graph convolutional network

Edit on GitHub

Note

Click [here](#) to download the full example code

Relational graph convolutional network

Author: Lingfan Yu, Mufei Li, Zheng Zhang

In this tutorial, you learn how to implement a relational graph convolutional network (R-GCN). This type of network is one effort to generalize GCN to handle different relationships between entities in a knowledge base. To learn more about the research behind R-GCN, see [Modeling Relational Data with Graph Convolutional Networks](#)

The straightforward graph convolutional network (GCN) and [DGL tutorial](#) exploits structural information of a dataset (that is, the graph connectivity) in order to improve the extraction of node representations. Graph edges are left as untyped.

A knowledge graph is made up of a collection of triples in the form subject, relation, object. Edges thus encode important information and have their own embeddings to be learned. Furthermore, there may exist multiple edges among any given pair.

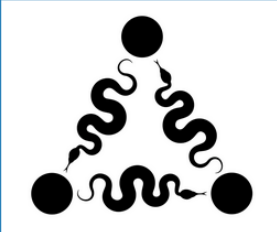
A brief introduction to R-GCN

In *statistical relational learning* (SRL), there are two fundamental tasks:

- **Entity classification** - Where you assign types and categorical properties to entities.
- **Link prediction** - Where you recover missing triples.

Source: https://docs.dgl.ai/en/0.4.x/tutorials/models/1_gnn/4_rgcn.html#implement-r-gcn-in-dgl





stable

GETTING STARTED

- Installation
- First Steps
- Tracking Results during Training
- Saving Checkpoints during Training
- A Toy Example with Translational Distance Models
- Understanding the Evaluation
- Optimizing a Model's Hyper-parameters
- Novel Link Prediction
- Running an Ablation Study
- Performance Tricks

BRING YOUR OWN

- Bring Your Own Data
- Bring Your Own Interaction

EXTENDING PYKEEN

- Extending the Datasets
- Extending the Models

[Home](#) » [Models](#) » RGCN [Edit on GitHub](#)

RGCN

```
class RGCN(triples_factory, embedding_dim=500, loss=None, predict_with_sigmoid=False,
           preferred_device=None, random_seed=None, num_bases_or_blocks=5, num_layers=2, use_bias=True,
           use_batch_norm=False, activation_cls=None, activation_kwargs=None, sparse_messages_slcwa=True,
           edge_dropout=0.4, self_loop_dropout=0.2, edge_weighting=<function inverse_indegree_edge_weights>,
           decomposition='basis', buffer_messages=True, entity_initializer='xavier_uniform', relation_initializer='uniform')
[source]
```

Bases: `pykeen.models.base._OldAbstractModel`

An implementation of R-GCN from [\[schlichtkrull2018\]](#).

This model uses graph convolutions with relation-specific weights.

See also

- Pytorch Geometric's implementation of R-GCN
- DGL's implementation of R-GCN

Initialize the module.

Parameters:

- triples_factory** (`TriplesFactory`) – The triples factory facilitates access to the dataset.
- loss** (`Optional[Loss]`) – The loss to use. If None is given, use the loss default specific to the model subclass.
- predict_with_sigmoid** (`bool`) – Whether to apply sigmoid onto the scores when predicting scores. Applying sigmoid at prediction time may lead to exactly equal scores for certain triples with very high, or very low score. When not trained with applying sigmoid (or using BCEWithLogitsLoss), the scores are not calibrated to perform well with sigmoid.
- preferred_device** (`Union[None, str, device]`) – The preferred device for model training and inference.
- random_seed** (`Optional[int]`) – A random seed to use for initialising the model's weights. **Should** be set when aiming at reproducibility.
- regularizer** – A regularizer to use for training.



Graph Neural Networks Theory to Practice

Emanuel Sallinger